[Hung Nguyen] C Programing Review

1.Passing Pointers as Arguments

When passing a pointer to a function, any changes made within the function directly affect the memory address pointed to by that pointer.

```
#include <stdio.h>
int a = 10;
void *p = &a;

void increment(int *a)
{
    *a += 3;
}

int main(void)
{
    increment(p);
    printf("%d", a);
    return 0;
}
```

```
13
```

If you pass the variable's value instead of a pointer, any changes made within the function will not affect the original variable's value.

```
#include <stdio.h>
int a = 10;
void *p = &a;

void increment(int a)
{
    a += 3;
}

int main(void)
{
    increment(a);
    printf("%d", a);
    return 0;
}
```

```
10
```

2.NULL pointer

If a pointer points to NULL, it means it holds a special value indicating that no object is being pointed to—not necessarily address 0x00. If we attempt to access the pointer's value, the result will be (nil).

```
int main(void)
{
    int *p = NULL;
    printf("%p", p);
    return 0;
}
```

```
(nil)
```

NULL is a special value that represents a "null pointer" - a pointer that does not point to anything. Trying to read or write to that address causes a crash or error(process canceled) because the operating system protects that zero address area.https://www.quora.com/What-does-a-null-pointer-point-to-if-it-does-not-point-to-anything-How-it-is-treated-internally.
It helps you avoid using pointers that are empty or invalid. You can compare a pointer to NULL to

check if it is safe to use.

```
int main(void)
{
    int *p = NULL;
    printf("%d", *p);
    return 0;
}
```

```
Segmentation fault (core dumped)
```

3.sizeof pointer

The size of a pointer depends on the operating system. For example, an integer pointer (`int*`) on a 64-bit operating system is 8 bytes in size. Why 8 bytes instead of 4 bytes (the size of an `int`)? Because a pointer's value is an address; on a 64-bit operating system, an address is 64 bits (8 bytes) long.

```
#include <stdio.h>

int main(void)
{
    int a = 6;
    int *p = &a;
    printf("%zu\n", sizeof(*p));
    printf("%zu\n", sizeof(p));
    return 0;
}
```

```
4
8
```

4.Array(a) and address of array(&a)

When declaring `int a[];`

- a is the starting address of the array.
- &a is the address of the entire array.

The relationship among `a`, `&a` and `&a + 1`

Assumption:

```
int a[4];
```

`a`:

```
a
|
▼
┌──────┬──────┬──────┬──────┐
| a[0] | a[1] | a[2] | a[3] |
└──────┴──────┴──────┴──────┘

↑      ↑
a      a + 1
```

`&a` is the address of **the entire array**:

```
&a
 |
 ▼
┌──────────────────────────┐
|    the entire array a    |
|         int [4]          |
└──────────────────────────┘

↑
&a
```

Therefore:

```
&a + 1
 |
 └──────────────────────────► after the entire array
```

With `a[4]`:

```
&a + 1
```

increase the address by the **size of the entire array**, i.e.:

```
sizeof(a) = 4 * sizeof(int)
```

While:

```
a + 1
```

just increase the address by:

```
sizeof(int)
```

For an integer array `a` with 5 elements, `a` represents the starting address of the array, and `a + 1` advances the address by 4 bytes; `&a` is the address of the entire array, and `&a + 1` advances it by 20 bytes. Example:

```
#include <stdio.h>

int main(void)
{
    int a[5];
    printf("%p\n", (void *)(a));
    printf("%p\n", (void *)(a + 1));
    printf("%p\n", (void *)(&a));
    printf("%p\n", (void *)(&a + 1));
    return 0;
}
```

```
0x7ffdf68691d0
0x7ffdf68691d4
0x7ffdf68691d0
0x7ffdf68691e4
```

5. Struct

If you want to assign and modify one or more characters in a string using a pointer within a struct, you can use an array and then assign it to that pointer.

```
#include <stdio.h>

typedef struct person
{
    char *name;
    int age;
    float height;
} person;

int main(void)
{
    person man;
    char name[] = "Nguyen Van Ba";
    man.name = name;
    man.age = 40;
    man.name[11] = 'K';
    printf("%s\n", man.name);
    return 0;
}
```

```
Nguyen Van Ka
```

What is the difference between directly assigning a "String" value to a pointer and assigning an array to a pointer?

By directly assigning the string `"Nguyen Van Ba"`, the pointer `ho_ten` points to a pre-existing string that cannot be modified.

```
char *name = "Nguyen Van Ba";
```

This method is not recommended when using pointers.

With the second method of assigning a value, the pointer `ho_ten` points to the address of an array; since an array is a variable located in writable memory, the values of its elements can be modified.

```
char name[] = "Nguyen Van Ba";
man.name = name;
```

There is something quite interesting about using character strings in a struct.
Unfortunately, you can't assign a value to an array like this:

```
#include <stdio.h>

typedef struct person
{
    char text[30];
    int a;
} person;

int main(void)
{
    person person1;
    printf("%zu\n", sizeof(person1));
    person1.text = "Hello world!";
    printf("%s\n", person1.text);
    return 0;
}
```

```
error: assignment to expression with array type
       person1.text = "Hello world!";
```

However, there is a solution for this! You can use the strcpy() function and assign the value to s1.myString, like this:

```
#include <stdio.h>
#include <string.h>

typedef struct person
{
    char text[30];
    int a;
} person;

int main(void)
{
    person person1;
    printf("%zu\n", sizeof(person1));
    strcpy(person1.text, "Hello World!");
```

```
    printf("%s\n", person1.text);
    return 0;
}
```

```
Hello World!
```

Size of Struct

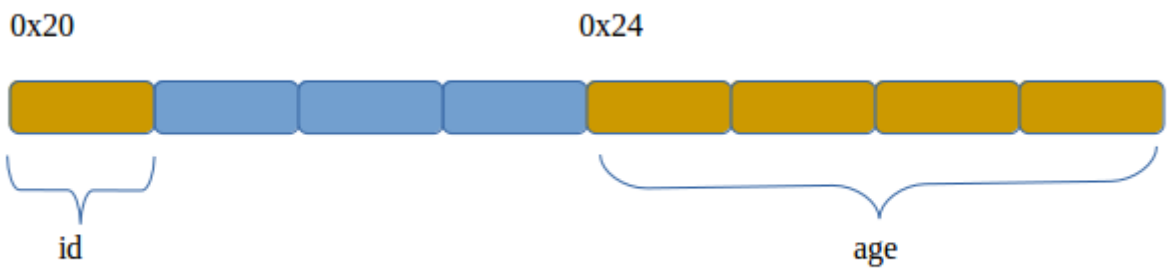
How the data of a struct in C or C++ is organized
Example:

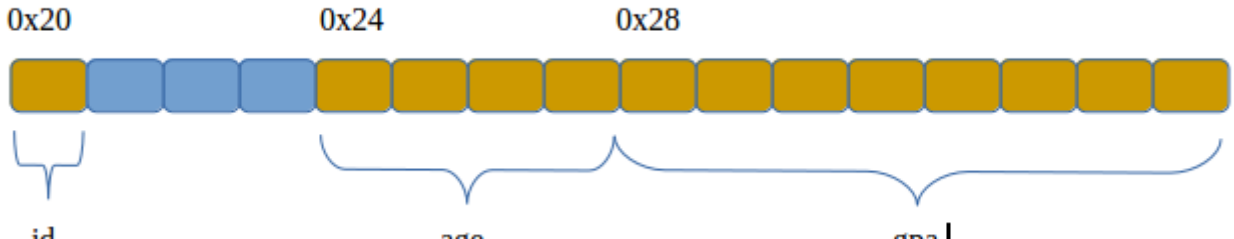
```
#include <stdio.h>

struct student
{
    char id;        /* 1 byte */
    int age;        /* 4 byte */
    double gpa;     /* 8 byte */
};

int main()
{
    printf("The size of Student: %d\n",sizeof(struct student));    /* print out 16 */
    return 0;
}
```

The compiler will determine that the `gpa` member of type `double` is the largest member within the struct (8 bytes), then allocate a block with a size based on the size of the largest member and populates the memory areas for the members according to their declaration order within the struct; when a block is exhausted, a new one is allocated.





For empty structs (those with no members), the compiler allocates one byte.

```
#include <stdio.h>

struct foo
{
};

int main()
{
    printf("%d\n",sizeof(struct foo));
    return 0;
}
```

```
1
```

The purpose of data padding is to ensure that data blocks align with even-numbered byte addresses, thereby increasing the CPU's memory read speed.

Compare two structs

Two structs may have identical members but contain different bytes in their padding.
Example two structs SensorData have the same member (id, distance, confident) but they don't have the same padding.
a:

id	00 00 00	distance	confidence	00 00 00

b:

id	FF FF FF	distance	confidence	AA BB CC

The members might be exactly alike, but:

```
memcmp(&a, &b, sizeof(SensorData))
```

may return a non-zero value.

6.Dynamic Memory

Notes on using Realloc()

`realloc()` changes the size of a memory block. Consequently, it may move the memory block to a different address. If `realloc()` fails, it returns: `NULL`.
In that case:

```
p = realloc(p, new_size);
```

causes p to lose the address of the old memory block.

```
Trước:
p
|
▼
0x1000
------------------
| old data       |
------------------
```

`realloc()` fall: `p = NULL` but the address range `0x1000` still exists and you loss thís address -> **Memory leak.** Safe method:

```
int *tmp = realloc(p, new_size);

if (tmp != NULL)
{
    p = tmp;
}
```

Forget to free() memory

Example:

```
void foo(void)
{
    int *p = malloc(100 * sizeof(int));

    p[0] = 10;
}
```

when foo() finished:

```
Stack:


p disappears
|
X


The heap remains allocated but cannot be accessed:


------------------------
| 100 int              |
------------------------
       ↑
       |
       No pointers pointing to it anymore.
```

Repeating this many times will consume a lot of RAM.

Using *p after free()

```
int *p = malloc(sizeof(int));


*p = 100;


free(p);


*p = 200;
```

The code still runs normally, but this is **unauthorized behavior.**

Dynamic with Struct

```
typedef struct
{
    char *name;
    int age;
    float height;
} person;
```

You can create a `person` on the Heap:

```
person *p = malloc(sizeof(*p));
```

Memmory:

```
Stack


p
|
|
▼

Heap


-----------------------------
| person                    |
|                           |
| name    | pointer         |
| age     | 25              |
| height  | 1.75            |
-----------------------------
```

But if:

```
p->name = malloc(100);
```

you have two Heap regions:

```
p
|
▼
Person
------------------
| name ----------|-------
| age            |      |
| height         |      |
------------------      |
                        ▼
                   ----------
                   | 100 B  |
```

```
                    ----------
```

When free() must:

```
free(p->name);
free(p);
```

If only `free(p);`, then the `p->name` region is leaked.

Double free()

If you free memory twice as shown below:

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int *p = malloc(sizeof(int));
    *p = 123;
    printf("%d\n", *p);
    free(p);
    free(p);
    printf("Hello!");
    return 0;
}
```

```
123
free(): double free detected in tcache 2
Aborted (core dumped)
```

The result is `123` and the program terminates; the string `Hello!` is not printed. This occurs because the code exhibits undefined behavior, leading to a crash and unpredictable errors.

How to avoid it?
Use `NULL` after free.

```
free(p);
p = NULL;
free(p);
```

7.Linked List

Some points to consider when using linked lists.

Memory leak

```
struct node *p = malloc(sizeof(struct node));


p = NULL;


free(p);
```

Initially:
p
↓

0x12345678

then:
p
↓
NULL

0x12345678

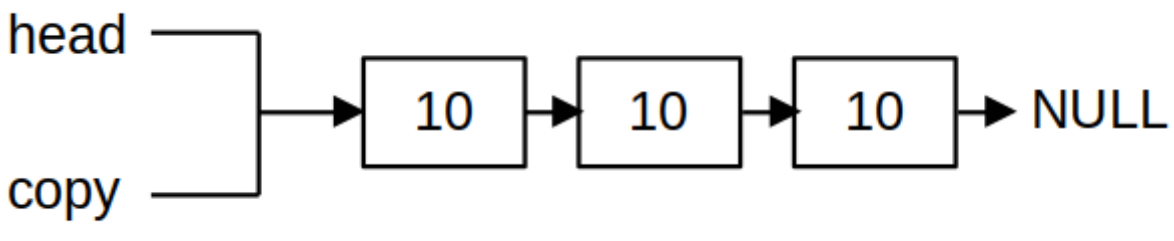
The memory region allocated by malloc() still exists, but you have lost the address used to access it.

Copy Linked List

If you use:

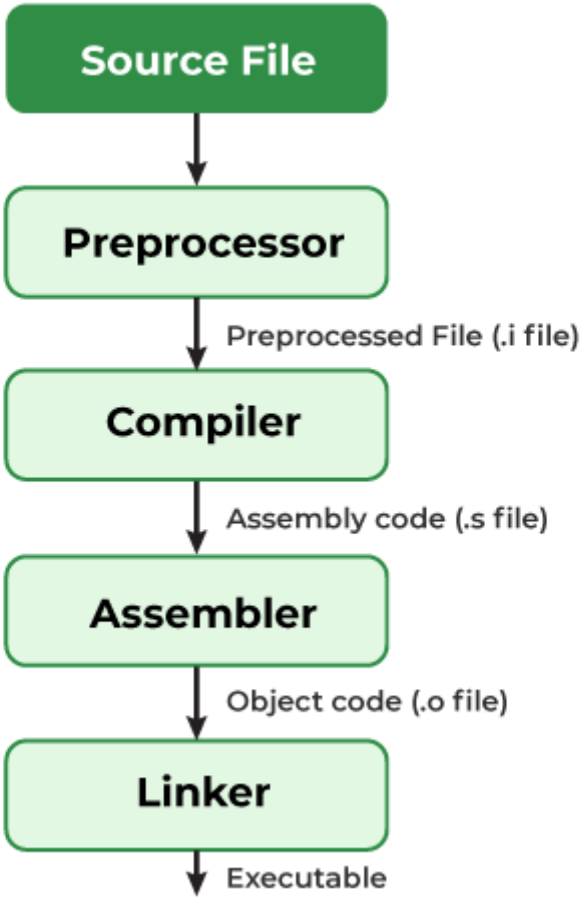
```
struct node *head = malloc(sizeof(struct node));
struct node *copy = head;
```

In reality, you are creating another pointer that also points to the memory area of `head`, rather than copying a new list.



If `copy->data = 999;` then the original list also becomes: `999`->`10`->`10`->NULL
To copy the list, you must allocate memory and copy each node within the list.

8.Compilation pipeline



9.Function pointers

10.Lab

Linked list

Link github: https://github.com/hungmanh150602/Zhone_Training/tree/main/session_2/linked_list

Compilation pipeline

Link github: https://github.com/hungmanh150602/Zhone_Training/tree/main/session_2/test_compile

Function pointer

link github:

https://github.com/hungmanh150602/Zhone_Training/tree/main/session_2/function_pointer

---

Revision #36
Created 2026-08-19 03:12:44 UTC by hung nguyen
Updated 2026-09-28 08:27:57 UTC by hung nguyen